DevOps Lab Assessment - Experiment 7

Docker Networking: Inter-Container IP & DNS Name Communication

STUDENT NAME:	**Rohit**	**ROLL NUMBER:**	**2301730179**
EMAIL ADDRESS:	rohit2301730179@gmail.com	**ASSESSMENT:**	Question 7 (5 Marks)
COURSE / LAB:	DevOps Lab 2 (Containerization & K8s)	**ENVIRONMENT:**	Docker 29 / Minikube v1.36 / kubectl

1. Experiment Objective & Problem Statement

Submitted by **Rohit** (Roll No: **2301730179**, Email: **rohit2301730179@gmail.com**). Containerized microservices must communicate reliably across isolated virtual networks. The objective of this experiment is to evaluate Docker networking modes, demonstrating: (1) the architectural limitations of the default bridge network regarding name resolution, (2) the creation of a user-defined bridge network, (3) inter-container communication using raw IP addresses, and (4) automatic service discovery and inter-container communication using container hostnames via Docker's embedded DNS engine.

2. Docker Networking Architecture & Drivers

Docker creates virtual Ethernet bridge interfaces (e.g. docker0) on the host. Crucially, the **default bridge** network does *not* provide automatic DNS resolution; containers can only reach each other if they know each other's ephemeral IP addresses. Conversely, **user-defined bridge networks** enable an embedded DNS server listening at 127.0.0.11:53, allowing seamless resolution of container names.

3. Experiment 7.1: Default Bridge vs Custom Bridge Networking

```
$ # 1. Run two containers on default bridge docker run -d --name c1-server alpine sleep 3600 docker run -d --name c2-client alpine sleep 3600 # 2. Ping by IP address (Expected: SUCCESS) docker exec c2-client ping -c 2 172.17.0.2 # 3. Ping by Container Name (Expected: FAILURE - no DNS) docker exec c2-client ping -c 1 c1-server
PING 172.17.0.2: 56 data bytes, 64 bytes from 172.17.0.2: seq=0 ttl=64 time=0.082 ms ping: bad address 'c1-server'
OBSERVATION: Name resolution failed as expected on default bridge!
```

4. Experiment 7.2: Custom Bridge Network & DNS Resolution

```
$ # Create custom network for candidate Rohit (2301730179) docker network create --driver bridge --subnet 172.20.0.0/16 lab-custom-net docker run -d --name backend-server --network lab-custom-net alpine sh -c \ "while true; do echo -e 'HTTP/1.1 200 OK\r\n\r\nHello from Backend Server' | nc -l -p 8080; done" docker run -d --name frontend-client --network lab-custom-net alpine sleep 3600
backend-server (attached to lab-custom-net, IP: 172.20.0.2) frontend-client (attached to lab-custom-net, IP: 172.20.0.3)
```

4.1 Communication via Raw IP vs Container Name

```
$ # Communication via Raw IP (Experiment 7.3): docker exec frontend-client wget -qO- http://172.20.0.2:8080 # Communication via Container Name (Experiment 7.4): docker exec frontend-client wget -qO- http://backend-server:8080 # Inspect Embedded DNS Resolution: docker exec frontend-client nslookup backend-server
Hello from Backend Server Hello from Backend Server Server: 127.0.0.11 Address: 127.0.0.11:53 Name: backend-server Address: 172.20.0.2
```

5. Engineering Observations & Conclusion

Key Technical Takeaways

1. **Candidate Identification:** Experiment executed by **Rohit** (Roll No: **2301730179**).
2. **Embedded DNS (127.0.0.11):** Custom networks activate Docker's internal DNS daemon, resolving container names dynamically without static IP hardcoding.
3. **Resilience to IP Churn:** If backend-server restarts and receives a new IP address, frontend-client continues communicating via the unchanging container name.